

Exploring Real-Time Satellite Computing: From Energy and Thermal Perspectives

Qing Li¹, Shangguang Wang¹, Chenren Xu², Xiao Ma^{*1}, Mengwei Xu¹,
Ao Zhou¹, Ruolin Xing¹, Boyuan Yang², Zuo Zhu¹, Ying Zhang², and Xuanzhe Liu²

¹Beijing University of Posts and Telecommunications, China

²Peking University, China

Abstract—Small satellites (SmallSats) are now widely used in various fields, such as real-time communication and earth observation. These increasingly complex space applications face limited support from conventional radiation-hardened processors onboard. Hence, many SmallSats are designed to utilize high performance commercial off-the-shelf (COTS) computing devices to address this problem but it remains unclear how the unique energy and thermal characteristics of SmallSats impact computing efficiency onboard. This work conducts a systematic and quantitative measurement study of COTS devices' computing efficiency on two real orbiting SmallSats. The key findings are: 1) inadequate energy management may lead to electricity wastage in sunlit zones and shortages in eclipse zones, impacting onboard computing availability and 2) the weak heat dissipation onboard may compromise COTS computing efficiency by incurring thermal throttling. To address such challenges, we design ProScale, a lightweight application-aware power management and thermal control system to improve computing efficiency under both electrical and thermal energy constraints. Evaluation shows that ProScale can improve the average task completion latency by $2.1\times$ for computation-intensive applications compared with baselines.

Index Terms—Satellite computing, energy-aware, thermal throttling, frequency scaling

I. INTRODUCTION

In the NewSpace era, Small Satellites (SmallSats) have dominated the space industry, constituting 72% of active satellites by 2023 [1]. They find applications in weather monitoring [2], real-time communication [3], and real-time Earth observation [4]. These increasingly complex space applications face limited support from conventional radiation-hardened processors onboard, lagging behind their terrestrial counterparts by roughly a decade in processor throughput [5]. Therefore, many SmallSats are designed to utilize cutting edge commercial off-the-shelf (COTS) computing systems, such as the Nvidia Jetson TX2 [6], Huawei Atlas 200 [7], Xilinx Zynq-7020 [8]. These COTS computing systems are characterized by low power consumption, high performance, and ease of system integration, making them highly suitable for onboard computing.

This work is supported in part by the National Science Foundation of China under Grants 62302015, 62425203, 62325201, 62032003, 62372061, U21B2016, and the Fundamental Research Funds for the Central Universities under Grant 2024ZCJH1. *Corresponding author: Xiao Ma (maxiao18@bupt.edu.cn).

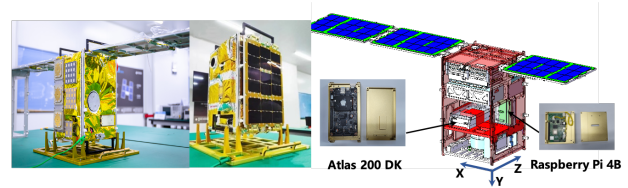


Fig. 1. Pictures of Baoyun and BUPT-1, inner structure of BUPT-1.

However, despite the popularity of COTS computing devices in space, it's crucial to note that SmallSats offer a complex working environment due to their limited size, weight, and power [9]. Specifically, the solar panel-battery alternating energy supply and passive thermal control design make computing on SmallSats challenging. On one hand, SmallSats rely on solar panels to harvest energy in the sunlit zones to power subsystems and charge batteries to keep functioning during the eclipse. The volume and mass constraints on solar panels and batteries, inherent to SmallSat design, pose a risk of unstable energy supply for computing applications. For example, a common category of SmallSats, CubeSats, weigh between 1~15 kg and have a volume of up to 12U (1U = 10 cm × 10 cm × 10 cm) [10]. Such CubeSats usually harvest power in the tens of watts range, with battery capacities in the dozens of watt-hours [9]. On the other hand, in contrast to ground-based systems using active cooling mechanisms, SmallSats primarily rely on passive thermal control methods, such as heat radiation and conduction, within a very compact structure [10]. The absence of heat convection in the vacuum of space exacerbates heat accumulation, which presents challenges for miniaturized COTS computing devices with high power density [11].

Traditional pre-launch practices, encompassing energy budgeting [12], thermal design [11], and simulated space tests, try to ensure SmallSats meet agency standards and are qualified for use in space applications. However, these practices have limitations in replicating real-world orbital operations. For example, energy budgets often disregard the dynamic nature of harvesting due to solar panel vibration during orbit [13]. Thermal ground tests cannot replicate thermal interactions among devices during satellite operations [14], [15]. This leads us to a fundamental question: “How close are COTS computing devices on orbit to realizing their potential as expected, and what steps are required to achieve this vision?”

Measurement. To answer this question, we conduct a comprehensive measurement study, assessing the performance of COTS computing devices in two real-world SmallSats, Baoyun and BUPT-1 (represented by Sat1 and Sat2), each with over 2 and 1 year of in-orbit operation, respectively¹. Our measurement involves Raspberry Pi 4B (RPI) [16] on both satellites and Atlas 200 DK (Atlas) [17] exclusively on Sat2 (refer to Fig. 1). We collect two datasets: one from Sat1, spanning 95 days, comprising telemetry and active experiment logs (8,000,000+ lines, 5.63 GB), and the other from Sat2, covering 123 days, with telemetry and logs (10,000,000+ lines, 15.61 GB). We have open-sourced the processed datasets on Github². Our measurement consists of two parts as described below.

Key Findings. Leveraging such unique datasets, we begin to measure how COTS computing devices behave under the solar panel-battery alternating power supply onboard. We find that the harvested power adequately meets the consumed power by computing devices during the short-term (tens of minutes) experiments on both Sat1 and Sat2. However, recent new real-time applications require continuous computing capabilities, such as the meet-up server on satellite for real-time multi-user interaction across continents [18] or the computing hardware for real-time object detection and tracking [19]. Therefore, we conduct a what-if analysis of long-term (tens of hours) computing on these two satellites. We find that the current harvested power in the sunlit zones is not enough, resulting in continuous battery discharge in the eclipse zones, depleting batteries within tens of hours. What's more, even with ample energy harvesting, i.e., nearly 90% power wastage during the whole simulation period (as in Table IV), it also leads to an accumulated discharge depth until battery depletion if consumed power in the eclipse zones exceeds the battery's maximum charging power limit in the sunlit zones.

Unlike terrestrial systems, where battery maintenance or replacement is feasible, a malfunctioning satellite battery renders the entire satellite unusable. To mitigate this, most satellites implement protective measures, allowing a maximum battery discharge depth (50% for both Sat1 and Sat2), to prevent severe lifespan degradation due to deep charge-discharge cycles. This protective strategy, however, triggers the forced shutdown of computing devices when the battery discharge exceeds the system limit. Therefore, it is critical to manage power usage onboard to avoid energy wastage in the sunlit zones while encountering energy shortages in the eclipse zones.

We then evaluate how COTS computing devices behave under the passive thermal design of SmallSats. We find that the thermal control system maintains the satellite's internal temperature within the normal working range of 1~30°C. However, RPIs on both Sat1 and Sat2 experience thermal throttling (with chip temperatures $\geq 80^\circ\text{C}$), causing up to a 2 $\times$ increase in computation latency. In contrast to terrestrial systems, where active thermal control (e.g., air or water

cooling) maintains optimal temperatures, it is challenging for SmallSats to achieve ideal thermal conditions for each device onboard. The compact structure and cost constraints limit the heat dissipation design space, including thermal pad material choice, contact area, and mounting position. Additionally, the overall satellite thermal design requires tradeoffs among different devices onboard. As a result, weak heat dissipation may compromise COTS computing efficiency by inducing thermal throttling. To address this issue, software-based thermal control mechanisms can be implemented to complement the deficiencies of device thermal design.

Enhancement. Facing these challenges, we investigate finer-grained dynamic power management and thermal control for COTS computing devices onboard to avoid forced shutdown or thermal throttling. Traditional ground-based systems use Dynamic Voltage and Frequency Scaling (DVFS) for power management and thermal control, which falls into the model-based method [20]–[23] and deep reinforcement learning (DRL)-based method [24] categories. Model-based approaches fail to create concise models in the dynamic satellite environment using constant parameters, while DRL methods, which adapt to dynamic environments, cannot effectively handle sudden energy supply variations—whether abundant or nonexistent—or drastic temperature changes (ranging from 0 to 30 °C as in Section II-C) as the satellite alternates between sunlit and eclipse zones periodically.

To overcome the aforementioned challenges, we propose ProScale, a lightweight, application-aware power management and thermal control system designed to enhance COTS computing efficiency in terms of task completion latency under both energy and thermal constraints. ProScale combines the simplicity of model-based approaches with the adaptability of DRL-based methods. It follows a two-stage design: offline orbit-aware profiling and online decision-making. In the orbit-aware profiling stage, lookup tables are constructed for each application profile, capturing power, temperature, and latency under different temperatures and frequencies in both sunlit and eclipse zones (simplicity). In the online decision-making stage, ProScale selects the appropriate operating frequency level based on these lookup tables. It then collects real-time data on power, temperature, and latency, updating the lookup tables accordingly (adaptability).

Real-world trace-driven experiment results indicate that ProScale achieves a significant reduction in average task completion latency by 2.1 $\times$ compared with the intermittent operation approach. Additionally, ProScale ensures no violations of battery discharge and temperature limits in the long run. ProScale is lightweight, causing only a slight overhead of less than 0.5°C temperature rise and less than 38.7mW power consumption, making it suitable for deployment on SmallSats.

II. A MEASUREMENT STUDY OF ENERGY-AWARE SATELLITE COMPUTING

This section investigates real-time satellite computing from the perspective of electrical energy and thermal energy on real orbiting SmallSats.

¹<http://www.tiansuan.org.cn>

²<https://github.com/qinglibupt/Satellite-computing-dataset>

TABLE I
SMALLSAT CATEGORIES.

Mass class name	Kilograms (kg)
Mini-satellites	100 – 180 kg
Micro-satellites	10 – 100 kg
Nano-satellites	1 – 10 kg
Pico-satellites	0.1 – 1 kg
Femto-satellites	0.001 – 0.1 kg

A. Measurement Setup

The real-world satellite-based experiment platform overview. SmallSats have broken records and are transforming in-space architectures. They are generally grouped according to their mass as shown in Table I [9]. One popular class of SmallSats is CubeSats, which adhere to a standard size and form factor [25]. CubeSats weigh only several to tens of kilograms and are in the unit of a 10 cm square cube (1U). Initially, CubeSats were composed of a single cube, known as 1U, but it is now common to combine multiple cubes to form larger configurations. Sat1 and Sat2 were launched on December, 2021, and January, 2023, respectively, with an expected lifespan of approximately three years. They currently orbit the Earth in a sun-synchronous orbit with an inclination of 97.4° at an altitude of around 500 km. Each orbital cycle lasts approximately 90 to 100 minutes, with about 66% of the time spent in the sunlit zone and the remainder in the eclipse zone.

SmallSats are typically equipped with two types of systems: *platform* and *payload*. The platform systems are responsible for enabling the satellites to function in space, providing support for electrical power, thermal control, communication, and propulsion. On the other hand, the payloads serve the primary mission objectives, such as cameras and computing devices. The satellite's electrical energy system is part of the platform and involves efficient and reliable energy generation, storage, and distribution to the on-board electrical subsystems. It consists of a solar module for energy harvesting and transfer, a battery module for energy storage, and a power distribution module to regulate the power supplied to the electrical subsystems [12]. Similarly, the thermal control system also belongs to the platform and is responsible for maintaining each subsystem within its nominal temperature limit throughout the satellite mission's entire lifespan [9]. Both RPis and Atlas are enclosed in custom aluminum housings, through which the heat is dissipated to the satellite structures and eventually released to space.

COTS Computing payloads and applications. We conduct active experiments on the COTS computing payloads, as shown in Fig. 1. Satellite computing holds great promise for satellite image processing using Deep Neural Network (DNN) models. Among these, on-orbit target recognition, such as identifying buildings, is a typical application. Therefore, we choose the YOLO model for image inference to perform target recognition. These inference models, along with the corresponding image and video datasets, are pre-deployed

TABLE II
APPLICATION AND DEVICES USED IN THE MEASUREMENT.

Models	Framework	Payload	Satellite
YOLOv3-tiny	Darknet [26]	RPi	Sat1
YOLO-fastest	MNN [27]	RPi	Sat2
YOLOv5-lite	MNN	RPi	Sat2
YOLOv3-544	MNN	RPi	Sat2
YOLOv3-416-1Thread	CANN [28]	Atlas	Sat2
YOLOv3-416-4Thread	CANN	Atlas	Sat2

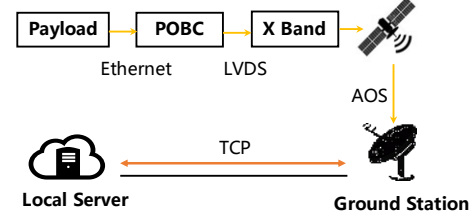


Fig. 2. Satellite data collection path.

before the satellite launch. The applications are summarized in Table II. We define several metrics to quantify the application performance, energy consumption, and temperature of onboard computing, which are summarized in Table III.

During the experiment, RPis utilize CPU resources for computation, while Atlas benefits from both CPU and NPU for accelerated inference. To observe the power and temperature at different frequency levels, we run the applications at varying frequencies one by one. RPis employ Linux CPU frequency management for system-level frequency scaling, whereas Atlas offers users four frequency levels: *Full*, *High*, *Middle*, and *Low*. Each experiment lasts for about 1-3 hours with 1-2 hours shutdown between experiments.

To give a more intuitive understanding of the thermal condition onboard the satellite, we develop a terrestrial testbed with the same devices and the same applications to conduct comparative experiments. In the terrestrial testbed, both RPis and Atlas are also enclosed in custom aluminum housings just like on the satellites as in Fig. 1. The RPi and Atlas are subjected to identical cooling conditions, maintaining the same fan speed under the same room temperature conditions.

Data collection. We collect two datasets from Sat1 and Sat2, each containing two parts: satellite platform telemetry data and payload experiment data. The telemetry data consists of information monitored by the satellite platform from various subsystems, such as the currents, voltages, and temperatures. The payload experiment data includes experimental logs based on COTS computing payloads, recording chip temperature, CPU frequency, CPU usage, inference latency, and more.

We collect the data via the satellite's X-band telemetry and telecommand link. Taking the payload experiment logs as an example, as shown in the Fig. 2, the payload, i.e., Atlas, utilizes the internal bus to send traffic to the Payload Onboard Computer (POBC) via Ethernet, which interfaces with the payloads. Subsequently, the packets are directed to the X-band module via the Low-Voltage Differential Signal (LVDS)

TABLE III
METRICS IN THE MEASUREMENT.

Metrics	Definition	Unit
Inference latency	Average per-frame image inference latency of the applications in Table II	s
Harvested/consumed power	The instantaneous power harvested by the solar panels and consumed by devices onboard	W
Underutilized energy	Energy that is neither consumed by the satellite's onboard devices nor charged into the batteries	J
Depth of Discharge (DoD)	Proportion of energy discharged from batteries	%
Ambient temperature	Temperature of the computing device surroundings	°C
Chip temperature	The processor chip's temperature of the COTS devices	°C
CPU Frequency	The CPU clock frequency of the COTS devices	Hz

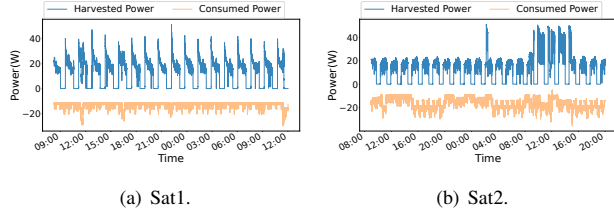


Fig. 3. Electrical energy harvesting and consumption trace.

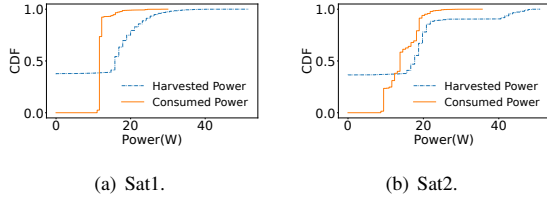


Fig. 4. Electrical energy statistics.

transmission. After modulation and amplification, the data is transmitted via the antenna to the ground station using the Advanced Orbiting Systems (AOS) protocol, and finally, it is sent to the local server on the ground.

B. Electrical Energy Impact on Satellite Computing

In this section, we try to answer two questions: 1) Is the energy supply enough in our onboard (intermittent) computing experiments? 2) Does the current electrical energy support continuous operation of computing devices? If not, what factors are the limitations?

Electrical energy harvesting and consumption overview. We perform computing experiments as in Table II. We first want to find if the harvested power can cover the consumed power onboard during our experiment. We get the harvested power and power consumed by all onboard electrical subsystems from the satellite telemetry. Fig. 3 illustrates the dynamics of harvested power and consumed power of Sat1 and Sat2 over multiple orbit cycles. The harvested power exhibits a distinct pattern of approximately 60 minutes of harvesting followed by 30 minutes of sleeping, recurring every 90 minutes while the consumed power onboard fluctuates randomly without an obvious pattern. Fig.4 quantifies the dynamics of harvested power and consumed power over the experiment period by illustrating the CDF. *Our analysis indicates that the harvested power is sufficient to cover the consumed power onboard during our experiments for both Sat1 and Sat2.*

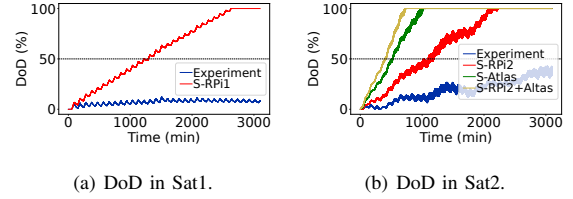


Fig. 5. Simulated continuous computing.

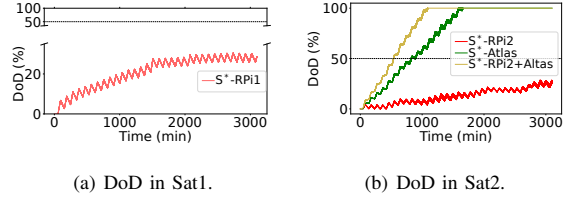


Fig. 6. Simulated continuous computing with augmented harvested power.

We further investigate how much harvested power is underutilized during the experiments in Table IV (Columns 1-2). Underutilized energy refers to the energy that is neither consumed by the satellite's onboard devices nor charged into the batteries. We calculate the percentage of time during the sunlit zones when the harvested power is underutilized, as well as the proportion of underutilized power to the harvested power. It can be observed that, for Sat1 (Sat2), 88.37% (49.28%) of the harvesting time has close to 40% of unused energy. *In our intermittent computing experiments, the satellite's energy is sufficient with a considerable degree of redundancy.*

Depth of battery discharge (DoD). If the computing devices on the satellite were to run continuously, akin to servers on the ground, can the current energy budget suffice? Conducting continuous computations on Sat1 or Sat2 bears a higher risk, such as the potential to deplete onboard energy, which could compromise the satellite's system safety. Therefore, we primarily rely on existing data for a what-if analysis. We need to analyze the DoD of the satellite battery, i.e., the proportion of discharged energy to battery capacity. If energy is sufficient, the DoD would not continuously accumulate before reaching 100%.

We first illustrate the actual DoD during the computing experiment on both Sat1 and Sat2 and then simulate the long-term continuous in-orbit operation of both RPi on Sat1 and RPi together with Atlas on Sat2 under the rated power (5 W for RPi and 12.8 W for Atlas). Fig. 5 reveals the energy balance

TABLE IV
UNDERUTILIZED ENERGY IN THE EXPERIMENT (E-) AND SIMULATIONS (S- AND S*-).

	E-Sat1	E-Sat2	S-RPi1	S-RPi2	S-Atlas	S-RPi2+Atlas	S*-RPi1	S*-RPi2	S*-Atlas	S*-RPi2+Atlas
Duration	95 days	123 days	50 hours	50 hours	50 hours	50 hours	50 hours	50 hours	50 hours	50 hours
Time (%)	88.37	49.28	7.75	13.57	10.24	6.00	99.00	99.37	99.17	99.19
Power (%)	37.29	41.87	19.24	34.671	15.38	6.45	90.53	89.13	85.11	82.48

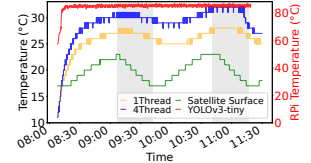
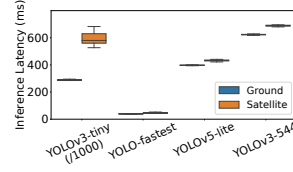
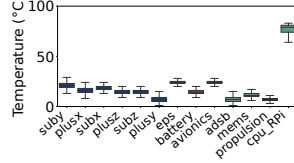
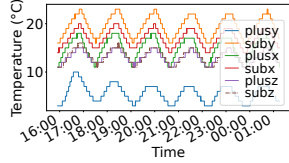


Fig. 7. Ambient temperature trace.

Fig. 8. Temperature overview.

Fig. 9. RPi's latency performance.

Fig. 10. Ambient temperature effects.

between supply and demand during the experiment. However, the current energy supply for the two satellites is insufficient to support continuous operation of the computing devices with DoD reaching 100% within tens of hours. DoD is a critical parameter that directly affects battery lifetime, with deeper discharge leading to a shorter lifetime in a logarithmic relationship [29]–[31]. Therefore, most satellites have protective design for battery to avoid deep discharge. The batteries on both Sat1 and Sat2 only allow a maximum discharge depth of 50% to maintain sustainable usage. Beyond this threshold, the satellite enters a minimum system operation mode, shutting down all the power-consuming payloads and keeping only essential platform systems for satellite functionality. The root cause of the computing devices' inability to remain continuously powered on is insufficient energy harvesting, as implied by the underutilized energy shown in Table IV (Columns 3-6). This results in the satellite battery not recharging, leading to a continuous increase in discharge depth in the eclipse zone until it surpasses the system limit.

To further investigate if continuous operation is feasible with sufficient harvested power, we augment the existing harvested power by 10 times and simulate continuous operation of the computing devices. It is evident that the harvested power is sufficient, i.e., nearly 90% underutilized energy during the whole sunlit zones in the simulation period as shown in Table IV (Columns 7-10). We observe two scenarios: one (of RPi on Sat1 in Fig. 6(a)) where the battery DoD balance in a range less than 50%, enabling continuous operation, and another (of RPi, Atlas, or both on in Fig. 6(b)) where the battery cannot be recharged, leading to an inability for continuous operation. These situations are due to the maximum charging current (power) limit of lithium batteries. For instance, the LG Chem INR21700-M50T battery used in the two targeted satellites has a maximum charging current of 1475mA [32]. If the power consumed in the eclipse zone can be replenished in the sunlit zone within the constraints of the maximum charging current, continuous operation of the computing devices is supported; otherwise, it is not. This situation leads to energy being wasted in the sunlit zone while being insufficient in the eclipse zone.

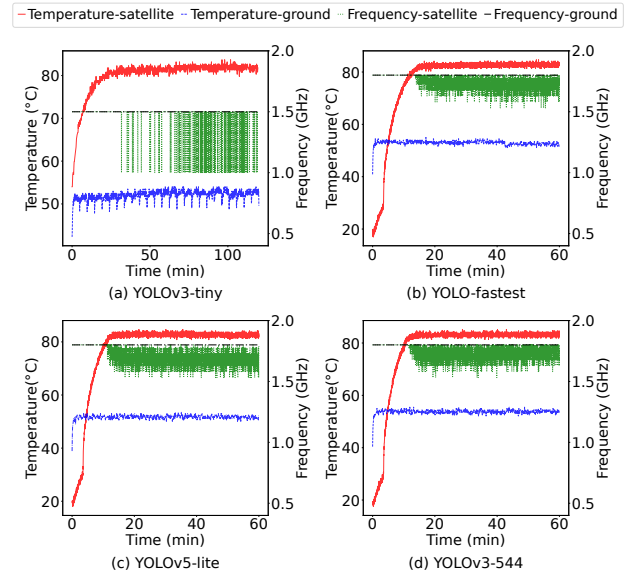


Fig. 11. RPi's thermal throttling: temperature and frequency.

Implications: These observations highlight the critical need for enhanced energy management and robust system design to ensure the reliability and longevity of satellite missions dependent on solar panel-battery power systems. Continuous monitoring and dynamic energy management are essential to adapt to fluctuations in energy availability due to factors like sunlit and eclipse zones. Balancing power use between sunlit and eclipse zones is crucial to avoid wastage and ensure an adequate energy supply. Besides, the risks associated with continuous computing on real satellites highlight the need for safety measures to prevent critical system failures due to energy shortages.

C. Thermal Energy Impact on Satellite Computing

This section investigates how the satellite's thermal control system influences the efficiency of COTS computing devices with high power density.

Temperature overview. The thermal control system maintains the satellite's internal temperature within the range

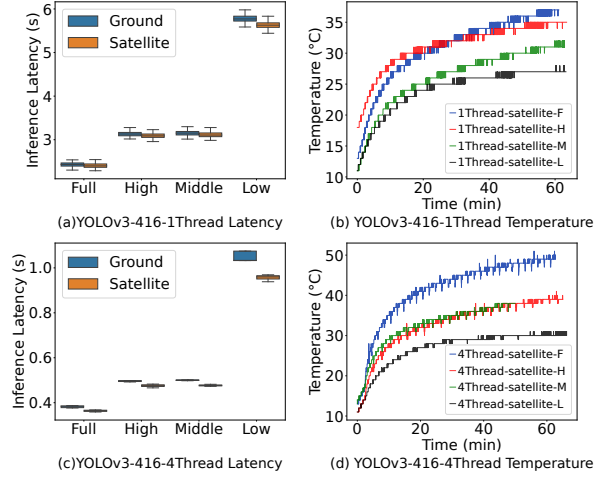


Fig. 12. Atlas without thermal issues: latency and temperature.

of 1~30°C, ensuring normal operation for non-computing subsystems. Fig. 7 displays the variations in temperature on the six cabin surfaces inside Sat1 over multiple orbit cycles. These variations occur with a period of around 90~100 minutes, as the satellite moves between sunlit and eclipse zones. SmallSats have relatively low thermal mass, making their temperature highly influenced by the environment. Fig. 8 presents the temperature distribution of the six cabin surfaces and onboard subsystems. The temperature range varies for each cabin surface, with the surface facing the Sun (labeled *suby*) consistently showing the highest temperature range of 13 to 35°C, while the opposite surface (*plusy* as in Fig. 1) remains the coldest at 1 to 15°C. The temperature variation of three platform components (i.e., EPS, battery, avionics) and four payloads is shown in Fig. 8. All subsystems remain within the normal temperature range of 0 to 30°C, except the RPi chip, which reaches 83°C. The heat dissipation is adequate for onboard subsystems but may be insufficient for the computing payload with high power density.

RPis' thermal performance. To have a more intuitive understanding of the thermal condition inside the satellites, we conduct the controlled experiments on the ground testbed. We control the heat dissipation between computing devices and aluminum housings, the hardware, the application, and the experiment time the same as the experiments on satellites. Then, we compare the inference latency of YOLO-based applications on RPis of Sat1 and Sat2 with those on the ground in Fig. 9. On both satellites, the inference latency increases up to 2× compared to the ground. Additionally, Fig. 11 shows the chip temperature and frequency variation of RPis over time. On the ground, the chip temperature stays in the range of 50~60°C with no overheating issues. However, on the satellites, the chip temperature increases over time, eventually stabilizing at around 80°C. This leads to thermal throttling and decreased computational efficiency.

Atlas's thermal performance. After identifying performance degradation in RPis onboard, we conduct experiments

on Atlas to observe its inference latency and chip temperature variation under satellite environments. These results are also compared to those obtained on the ground with the same active cooling condition of RPis. YOLOv3-416-4Thread has a lower inference latency than YOLOv3-416-1Thread at the cost of more CPU and NPU utilization. In Fig. 12 (a) and (c), we observe that Atlas exhibits no performance loss but even a slight improvement under both applications at each frequency level, compared to its performance on the ground. Fig. 12 (b) and (d) illustrate the chip temperature at both *Full* and *High* frequency levels. The chip temperature of Atlas on Sat2 is lower than when operating on the ground with active cooling, under both applications.

Thermal design variance analysis. The difference thermal performance indicates that Atlas has better heat dissipation compared to RPis on satellites. In the design phase, RPi and Atlas on Sat2 are positioned differently within the satellite, and their placement and thermal design vary as in Fig. 13. Both the Atlas and RPi are enclosed in custom aluminum housings and then mounted on the satellite structure. Specifically, the thermal path for the Atlas is as follows: the chip comes into thermal contact with the housing through a thermal pad, with a contact area of 2343 mm² per board; the mounting housing thermally contacts the satellite structure through a thermal film, with a contact area of 8214 mm². For the RPi, the thermal path involves the chip making thermal contact only with the mounting structure, and the mounting housing thermally contacts the satellite body through the structure, with a contact area of 7670 mm². It is noteworthy that the chip does not come into direct contact with the mounting housing through a thermal pad, which makes less efficient heat dissipation. Besides, other factors such as smaller contact area also contributes to the inefficient heat dissipation efficiency of the RPi compared to the Atlas.

Ambient temperature effects. The relatively low ambient temperature in eclipse zones cannot help cool down overheating chip temperature. In Fig. 10, we compare the temperature of RPi running at the default frequency mode and that of Atlas running at *Low* frequency level during two orbital cycles. The shaded regions represent the eclipse zones. Atlas chip temperature variation under both applications exhibits positive correlation with ambient temperature variation while RPi chip temperature is not affected by the ambient temperature variation because of chip overheating.

Implications: Thermal throttling poses significant challenges for computing payloads with high power density on SmallSats, and addressing this issue requires innovative and cost-effective thermal management solutions, such as new materials with superior thermal properties or advanced thermal management systems. In the absence of cutting-edge materials or advanced thermal systems, proper thermal conductivity design and strategic positioning of components become crucial during the initial design phase of the SmallSats. If the thermal issue still exists, we can leverage some software-based thermal control mechanisms to relieve the thermal issues.

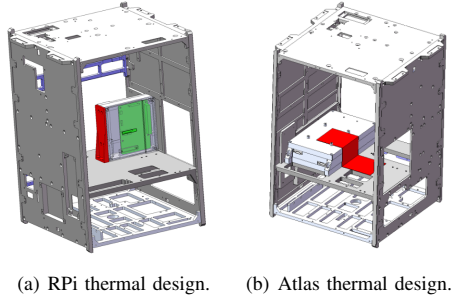


Fig. 13. Thermal design in Sat2.

III. APPLICATION-AWARE POWER MANAGEMENT AND THERMAL CONTROL

To enhance satellite computing efficiency in terms of task completion latency, we aim for finer-grained dynamic power management and thermal control capabilities, adapting to on-board energy constraints and the risk of overheating, rather than resorting to direct shutdowns or incurring thermal throttling. DVFS is a technique applied at the operating system level to reduce power consumption and maintain chip temperature within operational limits by dynamically adjusting the processor's voltage and frequency. In the following sections, we first explore how DVFS impacts application performance, power, and temperature. Then, we formulate the problem based on these observations and propose ProScale, a lightweight, application-aware power management and thermal control system based on the DVFS technique.

A. Finding Design Space

The DVFS experiment is conducted on the RPi of Sat1 and the Atlas of Sat2. We do not conduct the experiment on the RPi of Sat2 because both RPis exhibit similar power and temperature behavior onboard. On the RPi, although the frequency can be continuously adjusted between 0.6 and 1.5 GHz, the cost of in-orbit experiments is still very high. Therefore, we select three distinct frequency levels (0.6, 1, and 1.5 GHz) to balance experimental effectiveness with cost control. We choose a typical compute-intensive application, the YOLOv3-tiny image inference task, on the RPi of Sat1. On the Atlas, its system does not support direct user frequency adjustment, but instead offers four compute levels. Therefore, we directly use these four compute levels. On the Atlas, we select two typical image inference tasks: YOLOv3-416-1Thread (light load) and YOLOv3-416-4Thread (heavy load).

Latency, temperature, and power variations with frequency level. Figs. 14(a)-(c) show the latency, temperature, and power at different frequency levels. It is evident that DVFS has an impact on chip temperature, power, and latency across different hardwares and applications. We can observe that there are tradeoffs between frequency levels, chip temperature, power consumption, and latency. Different computing payloads may exhibit varying effectiveness in DVFS. For example, it is more effective in mitigating thermal throttling rather than

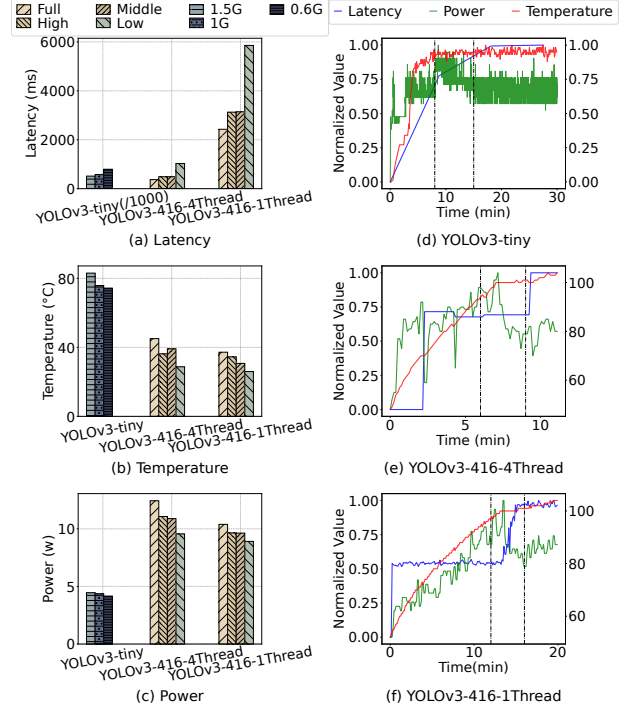


Fig. 14. Frequency scaling performance.

conserving energy for RPi. However, for the Atlas, which does not face thermal throttling issues, it is more impactful in saving energy. Compared to intermittent operation, dynamic DVFS is a worthwhile approach to explore, as it offers a finer-grained solution to alleviate both electrical and thermal energy bottlenecks.

Power and latency variations with temperature. Figs. 14(d)-(f) illustrate the changes in latency, power, and temperature at a fixed frequency (1.5 GHz for RPi, Full for Atlas). Previous measurements indicate that Atlas does not experience thermal throttling in orbit. To observe the effects of chip temperature on power and latency on the Atlas, we simulate a suboptimal heat dissipation environment on the ground, as shown in Figs. 14(e)-(f). To represent the three metrics in a unified manner, we employ the min-max normalization method, normalizing each metric as follows: $\text{normalized value} = (\text{value} - \min(\text{value})) / (\max(\text{value}) - \min(\text{value}))$. We emphasize the occurrence of thermal throttling in the areas enclosed by dotted lines of Figure 14(d)-(f). Power rises with higher chip temperatures at the same frequency level before thermal throttling. This is due to a feedback effect between electrical energy and thermal energy, where increasing temperatures raise leakage current, leading to higher power consumption, generating more heat, and further increasing the temperature [33]. The power drops when thermal throttling happens. As chip temperature increases, the inference latency remains relatively constant until chip overheating occurs, causing frequency throttling and an increase of inference latency.

B. Problem Formulation

We target the computation-intensive applications (e.g., DNN-based image processing applications) on SmallSats [34]. The applications run continuously to process the targeted tasks, such as objective detection [8]. ProScale operates in a discrete manner to make frequency scaling decisions, where each decision round, indexed by t , has a duration of τ (in seconds). For each application set A (including several applications running simultaneously) deployed on hardware H in each slot t , we need to decide how to scale the frequency level $x_t \in \{1, \dots, M\}$ (where M is the frequency level number, $t \in \{1, \dots, T\}$) to minimize the average latency of processing each image under both energy and thermal constraints.

Chip temperature model. The thermal conditions on the SmallSats are harsh, and chip temperature becomes a key factor affecting computing efficiency. At the end of each slot t , the chip temperature change $\Delta\Theta_t$ will vary depending on the applications, hardware, clock frequency, ambient temperature, and the chip temperature at the end of the previous slot. The intricate dependencies among these factors make it difficult to develop a concise mathematical model that accurately captures the accurate temperature change. We write the temperature change model as

$$\Delta\Theta_t = f(x_t, < A, H, \Theta_{t-1}, \Theta_{t,ambient} >). \quad (1)$$

Then, the temperature Θ_t at the end of each slot t can be expressed as $\Theta_t = \Theta_{t-1} + \sum_{i=1}^M \Delta\Theta_t$. To protect the chip and maintain the computing efficiency, the chip temperature cannot exceed the threshold $\Theta^{\max}$ in the long run, i.e.,

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T \Theta_t \leq \Theta^{\max}. \quad (2)$$

Electrical energy storage model. SmallSats have strict volume and mass constraints, which limit their energy harvesting and storage capacity. They use solar cells to harvest solar energy when in the sunlit zone, and rely on batteries in the eclipse zone. The energy stored in the battery can be formulated as a queue denoted by E_t^b ,

$$E_t^b = \min(\max(E_{t-1}^b + E_t^h - E_t^c, 0), E^{b,\max}), \quad (3)$$

where $\min(\cdot)$ and $\max(\cdot)$ are under the fact the harvested energy cannot be charged into a full battery anymore and battery cannot discharge after it has been fully drained. E_0 is the capacity of the full charged battery, E_t^h is harvested energy and E_t^c is the consumed energy, i.e., $E_t^c = (P_t^c + P_t^s)\tau$, where P_t^s is the sum power of all non-computing subsystems and P_t^c is the power of the computing payload under frequency level x_t , also influenced by application, hardware, chip temperature,

$$P_t^c = g(x_t, < A, H, \Theta_{t-1} >). \quad (4)$$

The DoD of battery is denoted by

$$D_t = 1 - E_t^b / E^{b,\max}. \quad (5)$$

It has a log relationship with the battery life, i.e., $\log_{10} Life + a \cdot D = b$, where a and b are constant parameters determined

by battery specifications. DoD is determined by the energy consumption onboard [29], [30]. To maintain a good battery usage manner, we limit the DoD to be smaller than $D^{\max}$ in the long-run, i.e.,

$$\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T D_t \leq D^{\max}. \quad (6)$$

For each application set A deployed on hardware H , tasks arrive continuously e.g., image processing tasks. We define the task completion latency in each time slot t as L_t ,

$$L_t = h(x_t, < A, H, \Theta_{t-1} >). \quad (7)$$

Our objective is to minimize the average task completion latency in the long run by dynamically scaling the frequency level x_t . The problem can be formulated as

$$\begin{aligned} \mathbf{P1} : \min_{\{x_t\}} \quad & \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=1}^T L_t \\ \text{s.t.} \quad & (2), (6). \end{aligned} \quad (8)$$

This problem cannot be directly solved by existing optimization methods for the following two reasons. First, the system dynamics in the future cannot be known in advance when we perform the frequency scaling. Second, the chip temperature change $\Delta\Theta_t$, power consumption P_t^c , and task completion latency L_t at frequency level x_t are all unknown.

C. Lyapunov Optimization Based Problem Transformation

We first leverage Lyapunov optimization theory [35] to decouple the long-term constraints and long-term objectives into real-time ones. To characterize the historical violations on both electrical and thermal energy constraints, we define virtual queues as,

$$Q_{t+1}^E = Q_t^E + \max\{D_t - D^{\max}, 0\}, \quad (9)$$

$$Q_{t+1}^T = Q_t^T + \max\{\Theta_t - \Theta^{\max}, 0\}, \quad (10)$$

where Q_t^E and Q_t^T are initialized as 0. According to the Lyapunov drift-plus-penalty framework in [35], the problem **P1** is transformed into

$$\mathbf{P2} : \min Q_t^E P_t^c \tau + Q_t^T \Delta\Theta_t + V L_t. \quad (11)$$

Till now, we have transformed the original long-term optimization problem **P1** to a real-time optimization problem **P2** in each time slot. We describe the online power management and thermal control algorithm in Algorithm 1. In each time slot t , it first solves the problem **P2** to obtain a frequency level, and then update the virtual queues. In problem **P2**, the frequency level number is small in practice (e.g., Atlas has only 4 levels) so that we use brute force search to find the best frequency level for the current time slot.

Input:

Output:

1: Initialization: Static look-up table construction. Monitor the chips initial temperature Θ_0 . Let $Q_0^E = 0$, $Q_0^T = 0$.

- 3: Look-up for the temperature change $\Delta\Theta_t$, the chip's power P_Θ , and the task completion latency L_t ;

- 5: Update the temperature change $\Delta\Theta_{t+1}$, the chip's power P_{t+1}^c , the task completion latency L_{t+1} ;

7: Update the length of virtual queues according to (9) and (10) based on $\mathbf{x}_t^*$.

1. Decide a frequency level

3-2. Update the virtual queue length

Frequency Scaler

Processor

Application

3-1. Update the table

Tables

Monitor

2. Collect the system state

Orbital-Aware Profiling

Dynamic Environment

At first glance, DRL-based methods seem ideal, as they are designed for dynamic and unknown environments. However, upon further investigation, we find that DRL performs well in consistent dynamic environments, but its performance declines when applied to a different environment, requiring model retraining [24]. The satellite environment, in terms of energy and thermal conditions, is inconsistent. For example, due to the satellite’s periodic movement, within a roughly 90-minute orbital cycle, about 60 minutes are spent in the sunlit zone, where energy is abundant and internal temperatures are high.

Look-up table tuning at runtime. Our current static profiling strategy overlooks dynamic changes in frequency

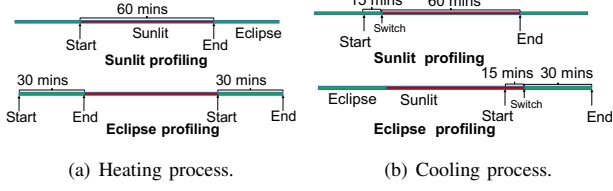


Fig. 16. Orbit-aware profiling illustration.

levels, resulting in estimation errors. To improve the accuracy of the look-up table, we collect the three parameters during the task execution and continuously perform a weighted average with the values in the table. Consider the heat dissipation characteristics, samples closer in time have a closer correlation with the current process's heat dissipation conditions. Therefore, One alternative is the exponential moving average weight method (temperature change $\Delta\Theta_i$ as an example),

$$\Delta\Theta_i = (\alpha\Delta\Theta_i + (1 - \alpha)\Delta\Theta_t)\mathbb{I}_{x_t=i}, \quad (12)$$

where α is the exponential moving parameter, $i \in \{1, \dots, M\}$ is any specific frequency level, and $\mathbb{I}_{(\cdot)}$ is an indicator function that equals 1 when the condition in the subscript is met, and 0 otherwise.

E. ProScale's Practical Usability

We highlight the practical usability of ProScale by further overhead analysis.

Algorithm overhead. The initial table construction involves two main phases: the data collection phase and the table construction phase. In the data collection phase, the profile automatically executes on the satellite, and a single frequency level requires at most two orbit periods. As the initial table construction is a one-time process, the table construction process has a time complexity of $O(M \cdot (\Theta^{\max} - \Theta^{\min}))$. The memory footprint of the generated table is small, in the order of $O(M \cdot (\Theta^{\max} - \Theta^{\min}))$.

The dynamic tuning incurs minimal overhead since temperature and power parameters are already part of the telemetry. We only additionally record the task completion latency. For time overhead, the reading of collected parameters into the lookup table and the table update time are negligible. For resource overhead, dynamic tuning does not expand the size of the static table which is very small at $O(M \cdot (\Theta^{\max} - \Theta^{\min}))$.

Safety consideration. ProScale can handle temporary energy shortages and prolonged overheating. It can include a 'zero' frequency level to deactivate the computing payload when the average DoD or chip temperature exceeds the threshold.

IV. IMPLEMENTATION AND EVALUATION

A. Experiment setup

The ideal scenario for system evaluation would be conducted on real satellites, but both Sat1 and Sat2 currently do not support the system design in ProScale. ProScale is deployed within the computational payload and requires access to telemetry data from the satellite platform, such as energy harvesting and the power consumption of other systems. This



Fig. 17. Terrestrial testbed.

is not supported in the current satellite design. However, we have included support for this requirement in the subsequent satellite design. Therefore, ProScale will be validated in the computational payload of future satellites. Although the different environments between the satellite and the ground testbed lead to variations in chip temperature, it does not affect system and algorithm operations.

We implement ProScale in Python 3.7 and evaluate it on the terrestrial testbed under both constrained electrical energy supply and thermal environments, as shown in Fig. 17. Two electrical energy supply scenarios are synthesized based on real energy harvesting and consumption data from Sat1. To observe ProScale's adaptability to energy shortages during the 195-minute experiment duration (equivalent to two orbital cycles), we simulate a small battery capacity of 29.5 Wh and set a relatively strict DOD threshold of 20%. Environmental temperatures, resembling those inside the satellite (experiencing sunlit and eclipse conditions), are simulated by adjusting the fan speed. Each simulated orbit cycle consists of 60 minutes with a low fan speed (sunlit) followed by 30 minutes with a high fan speed (eclipse). The chip temperature limit is set at 70°C. We use the same RPi and Atlas configurations on the ground to run the same applications as on satellites. For each hardware, we choose an application, YOLOv3-544 on RPi, and YOLOv3-416-4Thread on Atlas. The frequency levels for RPi and Atlas are 0.6, 0.9, 1.2, 1.5 GHz and Full, High, Middle, Low (labeled by 3, 2, 1, 0 in Fig.18), respectively. The metrics include task completion latency, chip temperature, and DoD, as discussed in Section II.

We compare ProScale with the following methods:

- *Default* [34] keeps the application running at the maximal frequency level without consideration on any system effect (such as power supply, chip temperature).
- *Intermittent Computing (IC)* [36] is similar to *Default* until it violates the temperature or energy constraints and turns off the computing payload to cool down or wait for sufficient energy.
- *Model-based* method [23] models the temperature and power consumption in Eqs. (1) and (4) and keeps the application running at the maximal frequency level under both temperature and energy constraints.

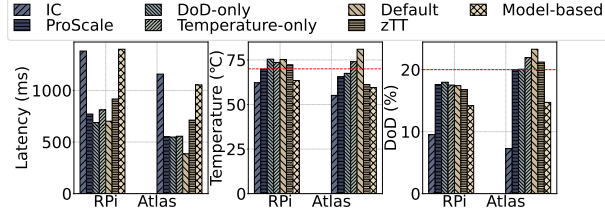


Fig. 18. Average performance comparison.

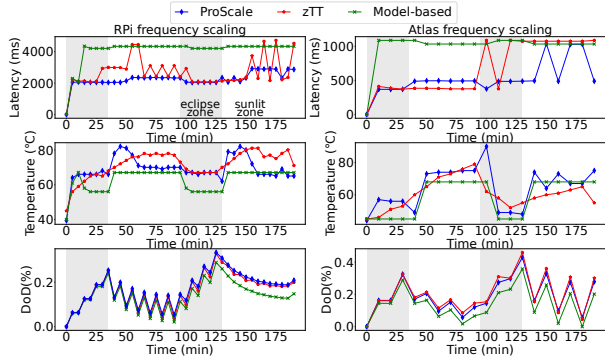


Fig. 19. Frequency scaling dynamics.

- *zTT* [24] is a recent DRL-based DVFS mechanism that aims to maximize computational performance while avoiding thermal throttling by learning the thermal environment changes of mobile devices.
- *DoD-only* is the same as ProScale but without considering the temperature constraint.
- *Temperature-only* is the same as ProScale but without considering the energy constraint.

Average performance comparison. ProScale enhances task completion latency, achieving up to $1.8\times$ and $2.1\times$ improvements for RPi and Atlas, respectively, without violating DoD and temperature constraints. Fig. 18 illustrates ProScale's performance in terms of average latency, chip temperature, and DoD for different devices and applications. ProScale efficiently adjusts frequency levels based on runtime state information, maximizing computing speed while meeting temperature and DoD constraints. *IC* is a conservative approach that avoids constraint violations but sacrifices computing efficiency. *Default* achieves minimal latency but disregards the risk of overheat damage and exceeds temperature limits. Model-based method has constant parameters that are derived under certain conditions. Thus, it is not adaptive to satellite environment dynamics and induces poor performance. *DoD-only* and *Temperature-only* are tailored versions of ProScale to fit in the condition with only DoD or temperature constraint. *zTT* exhibits longer average latency and violates temperature constraints under the periodic and drastic changes in energy and temperature.

B. Experiment results

Real-time dynamics. Fig. 19 illustrates the real-time variation of latency, chip temperature, and DoD for ProScale, model-based method, and *zTT*. Model-based method overestimates the temperature and power of the computing devices and scales the frequency to the lowest. Besides, *zTT* frequently adjusts compute levels, leading to significant fluctuations in latency. This is because it cannot adapt to the sudden (abundant or nonexistent) energy supply variations and drastic temperature changes (0-30 degrees) that occur as the satellite alternates between sunlit (60 minutes) and eclipse (30 minutes) zones periodically. In comparison, our method combines the strengths of high adaptiveness and low overhead. When the chip temperature or the DoD surpasses the specified threshold, ProScale reduces the frequency level. Conversely, as the chip temperature decreases or the energy supply is sufficient, ProScale increases the frequency level. Fig. 20 illustrates the proportion of different latencies associated with decisions relative to the total number of decisions during the experiment period. Under the constraints of DoD and temperature, ProScale's longest task completion latencies on RPi and Atlas are approximately 1000 ms and 350 ms, respectively, meeting the real-time requirements for Earth observation tasks [37].

Parameter impact. Fig. 21 illustrates the parameter impact. To represent the three metrics in a unified manner, we employ the maximum normalization method, normalizing each metric as follows: normalized value = value / max(value). It is crucial to choose an appropriate value for V that minimizes the average task completion latency while considering the electrical and thermal energy constraints. Fig. 21(a) shows the tradeoff between latency and energy/thermal cost, which is influenced by the parameter V . As V increases from 0.1 to 10, ProScale prioritizes latency performance, leading to a decrease in average latency. However, this emphasis on latency may result in higher average chip temperature and DoD, potentially exceeding the maximum budget limit. Fig. 21(b) depicts the influence of the decision round length on latency, chip temperature, and DoD. In real-world system operation, the decision round duration significantly impacts ProScale's performance. A shorter decision period allows for rapid adaptation to dynamic system changes. On the other hand, a longer decision period can achieve a balance between algorithm performance and minimizing overhead, such as system state information collection, especially when the system dynamics are relatively stable. Analyzing the system dynamics based on profile information is essential to determine the optimal decision round duration.

System overhead. ProScale is specifically designed to operate on SmallSats, which face both electrical and thermal energy constraints, making ProScale's overhead more pronounced compared to ground-based systems. We evaluate ProScale's overhead on both RPi and Atlas. The decision time is approximately 60~90 ms in each decision round. Moreover, ProScale incurs a temperature rise of less than 0.5°C , and

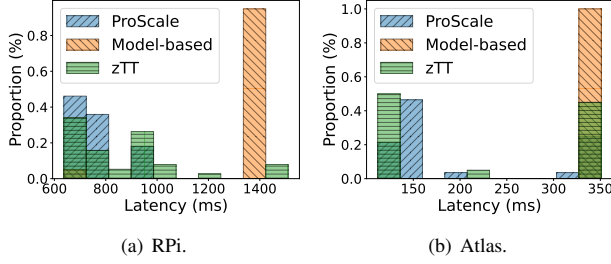


Fig. 20. Latency distribution.

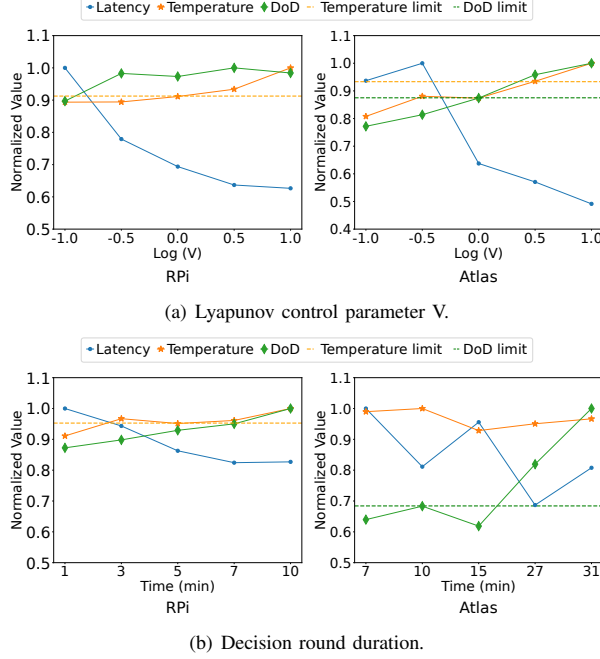


Fig. 21. Algorithm parameter impact.

consumes less than 38.7mW of power during continuous operation on both RPi and Atlas, demonstrating its suitability for computing on SmallSats.

V. RELATED WORK

Satellite computing. Since satellite computing is still in its early stages, various terms have been used to refer to it, such as satellite edge computing [38] and orbital edge computing [37]. In this paper, we adopt the unified term “satellite computing” to refer to this field. Early work in satellite computing focus on frameworks and applications. Denby *et al.* [37] propose support for computing in nano-satellite constellations and Casara *et al.* [39] explore the benefits of in-network computing in mega-LEO satellite constellations. Pfandzelter *et al.* [40] analyze different organization paradigms for satellite edge applications. Additionally, they investigate service placement and QoS constraints in satellite computing [41], [42]. Other studies propose resource deployment frameworks [43], computation offloading algorithms [44]. Space is the final frontier for computing and satellite computing is a research hotspot

full of opportunities and unique challenges. Denby *et al.* [34] explores the computational bottleneck of satellite computing and conclude that it lies in the inability to process all tiles within the frame deadline. From the systematic perspective, our findings indicate that the bottleneck of satellite computing lies in the limited electrical energy supply and passive thermal control. This results in a significant reduction in its computational efficiency, failing to achieve the performance observed in ground environments with abundant electrical power and efficient heat dissipation.

Frequency scaling for power and thermal management.

Previous work has utilized frequency scaling for power and thermal management. Gupta *et al.* [20] propose a power and thermal management approach based on frequency scaling, using a fitted model to estimate power budgets within temperature thresholds. Pathania *et al.* [21] and Prakash *et al.* [22] present thermal analysis models based on processor utilization to avoid thermal throttling during compute-intensive applications. Kim *et al.* [45] propose multi-core mapping and DVFS methods to ensure that the temperature of on-orbit computing devices in LEO CubeSats remains within an ideal operating range, thereby ensuring their reliability. Ouyang *et al.* [23] study a joint optimization approach for on-orbit computing and communication to minimize the end-to-end delay of data downlink from satellites. These works use power and temperature models based on fixed parameters to estimate the power consumption and temperature of computing devices. However, precise modeling of chip power and temperature in space is challenging due to numerous interconnected factors and the dynamic environment influence. Dynamic frequency scaling is a worthwhile technique to address both electrical and thermal energy bottlenecks on SmallSats. Kim *et al.* [24] propose a DRL-based frequency scaling technique to achieve the predefined frame rate of image processing applications while avoiding thermal throttling for mobile devices. It is responsive but performs poorly under the unique energy and temperature dynamics of satellites. As the satellite periodically enters sunlit and shadow zones, the drastically different environmental conditions lead to varying rewards for the model, ultimately causing confusion in the model’s decision-making. Therefore, we propose ProScale, which has the simplicity of modeling and the adaptability to unique environment changes.

VI. CONCLUSIONS

This work demystifies the impact of energy and thermal characteristics for computing on SmallSats, resulting from solar panel-battery alternating power supply and weak heat dissipation onboard. To address these challenges, we develop ProScale, a lightweight application-aware power management and thermal control systems, which significantly improves computing efficiency with no violations on battery discharge and temperature limits. Future work will focus on energy-aware inter-satellite collaboration to improve the computation efficiency of the whole satellite constellation.

REFERENCES

- [1] E. Kulu, "Small launchers - 2023 industry survey and market analysis," in *Proceedings of The International Astronautical Congress*, 2023, pp. 1–29.
- [2] Spire, "Unlock the potential of weather & climate analytics," 2023. [Online]. Available: <https://spire.com/weather/>
- [3] Z. Lai, W. Liu, Q. Wu, H. Li, J. Xu, and J. Wu, "Spacertc: Unleashing the low-latency potential of mega-constellations for real-time communications," in *Proceedings of IEEE Conference on Computer Communications*, 2022, pp. 1339–1348.
- [4] Planet, "Imaging the earth daily," 2023. [Online]. Available: <https://www.planet.com>
- [5] A. P. Arechiga, A. J. Michaels, and J. T. Black, "Onboard image processing for small satellites," in *Proceedings of IEEE National Aerospace and Electronics Conference*, 2018, pp. 234–240.
- [6] S. Sabogal, A. George, and C. Wilson, "Reconfigurable framework for environmentally adaptive resilience in hybrid space systems," *ACM Transactions on Reconfigurable Technology System*, vol. 13, no. 3, jul 2020. [Online]. Available: <https://doi.org/10.1145/3398380>
- [7] J. Murphy, J. E. Ward, and B. M. Namee, "Low-power boards enabling ml-based approaches to fdrr in space-based applications," in *Proceedings of Small Satellite Conference*, 2021, pp. 1–6.
- [8] V. Kothari, E. Liberis, and N. D. Lane, "The final frontier: Deep learning in space," in *Proceedings of the International Workshop on Mobile Computing Systems and Applications*, 2020, p. 45–49.
- [9] B. Yost, "State-of-the-art small spacecraft technology," 2023. [Online]. Available: https://www.nasa.gov/sites/default/files/atoms/files/2022_soa_full_0.pdf
- [10] A. Poghosyan and A. Golkar, "Cubesat evolution: Analyzing cubesat capabilities for conducting science missions," *Progress in Aerospace Sciences*, vol. 88, pp. 59–83, 2017.
- [11] M. F. Diaz-Aguado, J. Greenbaum, W. T. Fowler, and E. G. Lightsey, "Small satellite thermal design, test, and analysis," in *Proceedings of Modeling, Simulation, and Verification of Space-based Systems*, vol. 6221, 2006, p. 622109.
- [12] T. M. Lim, A. M. Cramer, J. E. Lumpp, and S. A. Rawashdeh, "A modular electrical power system architecture for small spacecraft," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 54, no. 4, pp. 1832–1849, 2018.
- [13] V. S. Aslanov and D. A. Sizov, "Chaos in flexible cubesat attitude motion due to aerodynamic instability," *Acta Astronautica*, vol. 189, pp. 310–320, 2021.
- [14] J. Miao, Q. Zhong, Q. Zhao, X. Zhao, J. Miao, Q. Zhong, Q. Zhao, and X. Zhao, "Design of spacecraft thermal control subsystem," *Spacecraft Thermal Control Technologies*, pp. 45–64, 2021.
- [15] V. Knap, L. K. Vestergaard, and D.-I. Stroe, "A review of battery technology in cubesats and small satellite solutions," *Energies*, vol. 13, no. 16, p. 4097, 2020.
- [16] Raspberry Pi, "Raspberry pi 4: Your tiny, dual-display, desktop computer," 2023. [Online]. Available: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>
- [17] Huawei, "Atlas 200 dk ai developer kit—huawei global," 2023. [Online]. Available: <https://e.huawei.com/en/products/computing/ascend/atlas-200>
- [18] D. Bhattacharjee, S. Kassing, M. Licciardello, and A. Singla, "In-orbit computing: An outlandish thought experiment?" in *Proceedings of ACM Workshop on Hot Topics in Networks*, 2020, p. 197–204.
- [19] Z. Lai, Q. Wu, H. Li, M. Lv, and J. Wu, "Orbitcast: Exploiting mega-constellations for low-latency earth observation," in *Proceedings of IEEE International Conference on Network Protocols*, 2021, pp. 1–12.
- [20] U. Gupta, R. Ayoub, M. Kishinevsky, D. Kadjo, N. Soundararajan, U. Tursun, and U. Y. Ogras, "Dynamic power budgeting for mobile systems running graphics workloads," *IEEE Transactions on Multi-Scale Computing Systems*, vol. 4, no. 1, pp. 30–40, 2018.
- [21] A. Pathania, A. E. Irimiea, A. Prakash, and T. Mitra, "Power-performance modelling of mobile gaming workloads on heterogeneous mpocs," in *Proceedings of the Annual Design Automation Conference*, no. 201, 2015.
- [22] A. Prakash, H. Amrouch, M. Shafique, T. Mitra, and J. Henkel, "Improving mobile gaming performance through cooperative cpu-gpu thermal management," in *Proceedings of the Annual Design Automation Conference*, 2016.
- [23] Q. Ouyang, N. Ye, J. Gao, A. Wang, and L. Zhao, "Joint in-orbit computation and communication for minimizing download time from leo satellites," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 3950–3963, 2024.
- [24] S. Kim, K. Bin, S. Ha, K. Lee, and S. Chong, "Ztt: Learning-based dvfs with zero thermal throttling for mobile devices," in *Proceedings of the Annual International Conference on Mobile Systems, Applications, and Services*, 2021, p. 41–53.
- [25] F. Davoli, C. Kourgiorgas, M. Marchese, A. Panagopoulos, and F. Patrone, "Small satellites and cubesats: Survey of structures, architectures, and protocols," *International Journal of Satellite Communications and Networking*, vol. 37, pp. 343–359, 2019.
- [26] DarkNet. [Online]. Available: <https://github.com/pjreddie/darknet>
- [27] MNN. [Online]. Available: <https://github.com/alibaba/MNN>
- [28] CANN. [Online]. Available: <https://www.hiascend.com/zh/software/cann>
- [29] Y. Yang, M. Xu, D. Wang, and Y. Wang, "Towards energy-efficient routing in satellite networks," *IEEE Journal on Selected Areas in Communications*, vol. 34, no. 12, pp. 3869–3886, 2016.
- [30] J. Fellner, G. Loeber, S. Vukson, and C. Riepenhoff, "Lithium-ion testing for spacecraft applications," *Journal of Power Sources*, vol. 119, pp. 911–913, 2003.
- [31] M. Lami, A. Shamayleh, and S. Mukhopadhyay, "Minimizing the state of health degradation of li-ion batteries onboard low earth orbit satellites," *Soft Computing*, vol. 24, no. 6, pp. 4131–4147, 2020.
- [32] Batemo Cell, "Lg chem inr21700-m50t data sheet," 2023. [Online]. Available: <https://www.batemo.de/products/batemo-cell-library/lg-chem-inr21700-m50t/>
- [33] H. Hanson, S. W. Keckler, S. Ghiasi, K. Rajamani, F. Rawson, and J. Rubio, "Thermal response to dvfs: Analysis with an intel pentium M," in *Proceedings of the International Symposium on Low Power Electronics and Design*, 2007, p. 219–224.
- [34] B. Denby, K. Chintalapudi, R. Chandra, B. Lucia, and S. Noghabi, "Kodan: Addressing the computational bottleneck in space," in *Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2023, p. 392–403.
- [35] M. Neely, "Stochastic network optimization with application to communication and queueing systems," *Synthesis Lectures on Communications*, vol. 3, no. 1, pp. 1–211, 2010.
- [36] G. Giuffrida, L. Fanucci, G. Meoni, M. Batič, L. Buckley, A. Dunne, C. van Dijk, M. Esposito, J. Hefele, N. Vercruyssen, G. Furano, M. Pastena, and J. Aschbacher, "The ϕ -sat-1 mission: The first on-board deep neural network demonstrator for satellite earth observation," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–14, 2022.
- [37] B. Denby and B. Lucia, "Orbital edge computing: Nanosatellite constellations as a new class of computer system," in *Proceedings of International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, p. 939–954.
- [38] Z. Zhang, W. Zhang, and F. Tseng, "Satellite mobile edge computing: Improving QoS of high-speed satellite-terrestrial networks using edge computing techniques," *IEEE Network*, vol. 33, no. 1, pp. 70–76, 2019.
- [39] P. Cassarà, A. Gotta, M. Marchese, and F. Patrone, "Orbital edge offloading in mega-geo satellite constellations for equal access to computing," *IEEE Communications Magazine*, vol. 60, no. 4, pp. 32–36, 2022.
- [40] T. Pfandzelter, J. Hasenburger, and D. Bermbach, "Towards a computing platform for the leo edge," in *Proceedings of the International Workshop on Edge Systems, Analytics and Networking*, 2021, p. 43–48.
- [41] T. Pfandzelter and D. Bermbach, "Celestial: Virtual software system testbeds for the leo edge," in *Proceedings of the ACM/IFIP International Middleware Conference*, 2022, p. 69–81.
- [42] Pfandzelter, Tobias and Bermbach, David, "Qos-aware resource placement for leo satellite edge computing," in *Proceedings of International Conference on Fog and Edge Computing*, 2022, pp. 66–72.
- [43] R. Xie, Q. Tang, Q. Wang, X. Liu, F. R. Yu, and T. Huang, "Satellite-terrestrial integrated edge computing networks: Architecture, challenges, and open issues," *IEEE Network*, vol. 34, no. 3, pp. 224–231, 2020.
- [44] M. Tong, X. Wang, S. Li, and L. Peng, "Joint offloading decision and resource allocation in mobile edge computing-enabled satellite-terrestrial network," *Symmetry*, vol. 14, no. 3, 2022.
- [45] B. Kim and H. Yang, "Temperature sensor assisted lifetime enhancement of satellite embedded systems via multi-core task mapping and dvfs," *Sensors*, vol. 19, no. 22, 2019. [Online]. Available: <https://www.mdpi.com/1424-8220/19/22/4902>